

This is the code base for the [ARC-AGI Without Pretraining](#) project. The Kaggle competition template version can be found [here](#).

Installation

```
> git clone https://github.com/iliao2345/CompressARC.git
> cd CompressARC
> python -m venv arc_agi_without_pretraining
> source arc_agi_without_pretraining/bin/activate
> pip install -r requirements.txt
```

How to solve an ARC-AGI task

Run `analyze_example.py` to initialize a new model and train from scratch:

```
> python analyze_example.py  
Enter which split you want to find the task in (training, evaluation, test): <split>  
Enter which task you want to analyze (eg. 272f95fa): <task>  
Performing a training run on task <task> and placing the results in <task>/  
|100%|███████████| 1500/1500 [12:22<00:00, 2.01it/s]  
done
```

The code will create a folder `<task>/` and put plots there after 1500 steps of training:

- solutions at every 50 steps
- interpretable tensors of task representations
- graph of each tensor's contribution to the KL over time
- graph of the KL vs reconstruction error over time

Most tasks may take up to 20 minutes to run, on one NVIDIA GeForce RTX 4070 GPU.

How to see which puzzles were solved, using the run information in this repo

```
Run python list_solved_puzzles.py results_for_the_blog_post/predictions_training.npz training 2000
```

This prints out a table indicating

- names of solved puzzles
- how many guesses would have been required (up to 4000) to solve each puzzle
- total number of solved puzzles with 2 guesses
- total number of solved puzzles

You can change the 2000 to any number of inference-time steps, if you want to observe partway through the solution process. You can also change `predictions_training.npz` and `training` to `predictions_evaluation.npz` and `evaluation` to see which evaluation puzzles were solved as well. If you want the results in a text file instead, you can pipe the output into a file:

```
python list_solved_puzzles.py results_for_the_blog_post/predictions_training.npz training 2000 >> result.txt
```

Running All Tasks and Debugging Guide

Running all tasks in one command

Two scripts already support running the full training split (400 tasks) in a single command. They differ only in execution strategy — the algorithm, number of iterations (2000), and optimizer settings are identical for every task.

Sequential — train.py

```
> python train.py
```

Trains all 400 tasks **one at a time**. Safe and simple, but slow.

Property	Value
Wall-clock time	~130 h (400 × ~20 min)
GPU VRAM needed	~0.5–1 GB (one task at a time)
Output	<code>timing_result.txt</code> , per-task plots, rolling <code>predictions_training.npz</code>
Accuracy saved?	Yes — updates after every task finishes

Parallel — `parallel_train.py` (recommended)

```
> python parallel_train.py
```

Runs as many tasks simultaneously as your GPU VRAM allows, then refills slots as tasks finish. 4–10× faster than sequential.

Property	Value
Wall-clock time	~15–30 h
GPU VRAM needed	up to full GPU memory (scheduler auto-limits)
Output	<code>submission.json</code> (Kaggle-format)
How it works	Phase 1: runs 2 iterations per task to measure VRAM footprint. Phase 2: greedy scheduler packs tasks onto the GPU under the safe memory budget.

Changing the split

Both scripts hardcode `split = "training"` near the top of their `__main__` block. To run evaluation or test tasks, edit that line:

```
# in train.py or parallel_train.py, find this line and change it:
split = "training" # change to "evaluation" or "test"
```

Are results the same between the two scripts?

Statistically equivalent, but **not bit-for-bit identical**. Both run the same algorithm for 2000 iterations per task.The differences are:

- **Random initialization:** `train.py` advances a single global seed (set once to 0) sequentially across all tasks. `parallel_train.py` spawns each task in a fresh subprocess; each subprocess re-imports `arc_compressor.py`, which resets the seed to 0 — so parallel tasks start from approximately the same seed and diverge only from different task architectures.
- **GPU non-determinism:** when multiple tasks share the GPU concurrently, floating-point operation ordering is not guaranteed.

The expected **pass@2 rate across the full split is the same** (~34.75 % on training). Individual task solutions may differ between runs.

Step-by-step debugging guide

GPU required. All scripts call `torch.set_default_device('cuda')` at import time. Running on a machine without a CUDA GPU will fail immediately. Verify with: `python -c "import torch; print(torch.cuda.is_available())"`.

Use `analyze_example.py` as the debug entry point — it runs a single task interactively, produces plots, and is the easiest place to insert breakpoints.

```
> python analyze_example.py
Enter which split you want to find the task in (training, evaluation, test): training
Enter which task you want to analyze (eg. 272f95fa): 272f95fa
```

Some known-interesting tasks to start with: `272f95fa`, `6d75e8bb`, `6cdd2623`, `41e4d17e`, `2bee17df`.

Execution flow and what to inspect at each step

Step 1 — Task preprocessing (`preprocessing.py`)

```
task = preprocessing.preprocess_tasks('training', ['272f95fa'])[0]
```

Key attributes to inspect:

Attribute	What it tells you
<code>task.n_examples</code>	Total examples (train + test pairs)
<code>task.n_train, task.n_test</code>	Split counts
<code>task.shapes</code>	<code>[[in_shape, out_shape], ...]</code> per example. Check that output shapes are predicted correctly for test examples.
<code>task.n_colors</code>	Number of non-black colors found in the task
<code>task.in_out_same_size</code>	If <code>True</code> , output grid = input grid size (simplifies loss)
<code>task.problem</code>	Tensor <code>[n_examples, n_x, n_y, 2]</code> — channel 0 = input, channel 1 = output

Step 2 — Model creation (`arc_compressor.py`)

```
model = arc_compressor.ARCCompressor(task)
print(sum(w.numel() for w in model.weights_list))  # should be ~76 000
```

- `model.weights_list` — shared model weights (~76 K params). These are passed to the optimizer.
- `model.multiposteriors` — task-specific latent means/variances. **Not** in `weights_list`; their shape scales with `n_examples × n_colors × n_x × n_y`.

Step 3 — Optimizer

```
optimizer = torch.optim.Adam(model.weights_list, lr=0.01, betas=(0.5, 0.9))
```

`betas=(0.5, 0.9)` is intentionally non-standard: lower β_1 makes the optimizer react faster to gradient changes on small, task-specific datasets.

Step 4 — Logger (`solution_selection.py`)

```
logger = solution_selection.Logger(task)
```

Tracks two pass@2 candidates per step: the current VAE sample and an EMA (decay=0.97) of logits. After training, `logger.solution_most_frequent` and `logger.solution_second_most_frequent` are the final predictions.

Step 5 — Training step (`train.py` → `take_step`)

```
train.take_step(task, model, optimizer, train_step=0, train_history_logger=logger)
```

The forward pass returns:

Variable	Shape / type	What to watch
<code>logits</code>	<code>[n_ex, n_colors+1, n_x, n_y, 2]</code>	Color predictions for input/output grids
<code>x_mask, y_mask</code>	<code>[n_ex, n_x/n_y, 2]</code>	Grid boundary predictions
<code>KL_amounts</code>	list of 27 tensors	One KL value per multitensor component
<code>KL_names</code>	list of 27 strings	Which dims each KL corresponds to, e.g. <code>"[1, 1, 0, 1, 1]"</code>
<code>total_KL</code>	scalar	Should decrease over training
<code>reconstruction_error</code>	scalar	Should decrease over training

Step 6 — Monitor posterior collapse

The most common failure mode is **posterior collapse**: 14 of the 27 multitensor components drop to $KL \approx 0$ and never recover, meaning the model stops using those information channels.

```
# After a training step, print per-tensor KL values:
for name, kl in zip(KL_names, KL_amounts):
    print(f"{name}: {kl.sum().item():.4f}")
```

If nearly all values are near zero after step 50–100, the run is unlikely to find a solution. Re-run with a different seed or see the architecture docs for the free-bits mitigation proposal.

Common failure causes

Symptom	Likely cause
<code>RuntimeError: CUDA error</code> or <code>AssertionError</code> on import	No CUDA GPU available. All modules set <code>torch.set_default_device('cuda')</code> at import time.
Results differ across runs with same seed	Known issue — noted in <code>analyze_example.py</code> comments. Seeds don't fully fix results.
Loss stays high / all KL near zero after ~100 steps	Posterior collapse. Normal for many tensors to collapse; problematic if all do.
<code>grid_size_uncertain = True</code> in loss loop	Output size prediction uncertain (neither same-size nor all-same-output-size). The loss uses a curriculum coefficient for the first 100 steps to stabilize.
<code>submission.json</code> missing some task keys after <code>parallel_train.py</code>	A subprocess crashed silently. Check <code>error_queue</code> propagation or run <code>python train.py</code> for a sequential run with full tracebacks.

Tips for Reading the Code

A basic description of the code files in this repo:

For running via command line:

- `analyze_example.py`: Demonstrates how to solve one ARC-AGI problem using our method, with visualizations of learned task representations and plots of metrics.
- `plot_problems.py`: Plots all of the ARC-AGI problems in a split.
- `plot_accuracy.py`: Plots pass@n accuracies during/after a bulk training run with `train.py`.
- `train.py`: Trains a model for every task in a split, plotting the accuracy. Contains code that computes the loss function. Defaults to the training split.
- `parallel_train.py`: A multiprocessing program that schedules as many puzzles as possible in a split to be solved at the same time through `solve_task.py`, while maximizing the GPU memory usage. Defaults to the training split.
- `scoring.py`: A script for scoring the results of `parallel_train.py`, which are better-formatted for Kaggle submissions.

Functionality, not for running via command line:

- `arc_compressor.py`: The network architecture and forward pass.
- `initializers.py`: Model initialization, and handling of equivariances via weight tying.
- `layers.py`: Implementation of individual layers in the forward pass.
- `multitensor_systems.py`: Handling multitensors.
- `preprocessing.py`: Converting the dataset into a form usable by the repo.
- `solution_selection.py`: Logging metrics and converting model outputs into solution predictions.
- `visualization.py`: Drawing problems and solutions.
- `solve_task.py`: A job script to solve one ARC-AGI problem on a specified GPU.

Some classes that the repo defines and uses:

- `MultiTensorSystem` (in `multitensor_systems.py`): A class that can spawn MultiTensors using stored dimensional information.
- `MultiTensor` (in `multitensor_systems.py`): Container class for groups of tensors.
- `Logger` (in `solution_selection.py`): For postprocessing of solutions outputted by the model, and their collection over time during training.
- `Task` (in `preprocessing.py`): Contains information about an ARC-AGI task, such as grid dimensions and masks, pixel colors, etc.
- `ARCCompressor` (in `arc_compressor.py`): Model class, with forward pass.
- `Initializer` (in `initializers.py`): For initializing model weights.

Some repo-specific language that we use for variable naming, etc.

- `dims` refers to a length 5 list of zeros and ones, and refers to the presence/absence of each of the five multitensor dimensions (*example, color, direction, height, width*). Channel dimension is implicitly included.
- `axis` always refers to the index of some dim in a tensor. For example, in a (*example, color, height*) tensor, the *height* dim is the 2nd axis, whereas for the (*height, width*) tensor, it is the 0th axis.
- This repo uses `x` and `y` to refer to the *height* and *width* dimensions, respectively.
- The `@multitensor_systems.multify` decorator takes a function and modifies it to apply it once for every tensor in a multitensor. If the input is a tensor/object, then the new input is now a multitensor/multiobject. The function must be written with additional parameter `dims`.
- The `@layers.add_residual` decorator takes a function and creates a residual connection around it, with projections to/from the input/output of the function and the residual stream. Optional parameters are added for using biases for the projections, using pre-norm, and post-norm.
- The `@layers.only_do_for_certain_shapes(*shapes)` decorator takes a function with `dims` as its first input, and applies the function only if `dims` is in `shapes`. Else, it applies the identity function. Useful for chaining with the `@multitensor_systems.multify` decorator.

Code for different files may be written in slightly different styles due to polishing of individual code files by ChatGPT.

Citation

If you'd like to cite this blog post, use the following entry:

```
@online{liao2025arcagiwithoutpretraining,
  author = {Isaac Liao and Albert Gu},
  title = {ARC-AGI Without Pretraining},
  year = {2025},
  url = {https://iliao2345.github.io/blog_posts/arc_agi_without_pretraining/arc_agi_without_pretraining.html},
}
```

Feature 1: Cached VRAM calculations

Updated todo list

Good idea. The VRAM footprint per task depends only on the task's grid dimensions/colors (fixed in the dataset) and the GPU model. It will always produce the same result on the same hardware, so it's safe to cache.

Let me implement it — two helper functions + a small change to the Phase 1 block in `run_split()`:

Searched for text `Per-split runner (**/parallel_train.py)`, 1 result

Read [](file:///home/asier/repos/arc-agi/src/SuperCompressARC/parallel_train.py#233-233), lines 233 to 300

Now I'll add the two cache helpers before `run_split` and replace the Phase 1 block:

Replace String in File

Multi-Replace String in Files

Multi-Replace String in Files

Created 3 todos

File editing tools are currently disabled. I'll use a Python script via terminal to apply the changes:

Run in Terminal

File editing and terminal tools are not available in Plan mode. Let me present the exact changes needed so you can apply them (or switch to implementation mode).

Updated todo list

Voy a leer los archivos clave para dar una explicación precisa y fundamentada en el código real.

Read [](file:///home/asier/repos/arc-agi/src/SuperCompressARC/train.py#1-1), lines 1 to 145

Read [](file:///home/asier/repos/arc-agi/src/SuperCompressARC/arc_compressor.py#1-1), lines 1 to 80

Read [](file:///home/asier/repos/arc-agi/src/SuperCompressARC/solution_selection.py#1-1), lines 1 to 100

Read [](file:///home/asier/repos/arc-agi/src/SuperCompressARC/preprocessing.py#1-1), lines 1 to 100

Tengo todo el contexto necesario. Aquí va la explicación paso a paso, con base teórica:

Paso a paso del núcleo de CompressARC (líneas 38–78)

Paso 0 — Configuración del entorno (`analyze_example.py`)

```
split = 'training'
task_name = '6d75e8bb'
folder = 'results/' + task_name + '/'
os.makedirs(folder, exist_ok=True)
```

Define el puzzle a resolver y el directorio de salida. CompressARC entrena **un modelo independiente por puzzle**, desde cero, sin ningún pretraining previo. Esto materializa el principio MDL (Minimum Description Length): la "inteligencia" está en el programa más corto que explica los ejemplos, no en conocimiento transferido.

Paso 1 — Preprocesamiento: `preprocessing.preprocess_tasks` (`analyze_example.py`)

```
task = preprocessing.preprocess_tasks(split, [task_name])[0]
```

Convierte el JSON crudo de ARC-AGI en un objeto `Task`. Las operaciones clave dentro de `preprocessing.py`:

Atributo producido	Descripción
<code>task.n_train, task.n_test</code>	Nº de pares demostración / test
<code>task.shapes</code>	<code>[[in_shape, out_shape], ...]</code> por ejemplo. Para test, el <code>out_shape</code> se <i>predice</i> por heurística: si todos los outputs de train son iguales, usa esa forma; si <code>input==output</code> en train, asume igual; si no, <code>max(n_x, n_y)</code>
<code>task.colors,</code> <code>task.n_colors</code>	Colores únicos encontrados; negro siempre incluido como fondo
<code>task.problem</code>	Tensor <code>[n_examples, n_x, n_y, 2]</code> — canal 0=input, canal 1=output
<code>task.multitensor_system</code>	Instancia de <code>MultiTensorSystem(n_examples, n_colors, n_x, n_y)</code> — define los 27 tensores válidos

Base teórica: el multitensor separa las 5 dimensiones fundamentales de ARC: (*examples, colors, directions, x, y*). Son los **Core Knowledge priors** de Chollet (2019, §III.1.2) hardcodedos: espacio 2D, colores anónimos, ejemplos múltiples.

Paso 2 — Creación del modelo: `ARCCompressor` (analyze_example.py)

```
model = arc_compressor.ARCCompressor(task)
```

Instancia el decodificador VAE (~76.000 parámetros). En `arc_compressor.py` se inicializan:

```
multiposteriors      → medias μ del posterior q(z|x) (una por cada tensor del multitensor)
decode_weights       → proyección latente (dim=4) → residual stream (dim=8 o 16)
target_capacities    → capacidad de canal por tensor (cuántos bits puede transmitir)
share_up/down_weights, softmax_weights, cummax_weights, shift_weights,
direction_share_weights, nonlinear_weights → 4 bloques de capas
head_weights, mask_weights → cabezas de predicción de colores y tamaño de grid
```

Base teórica (VAE, Kingma & Welling 2014): en un VAE estándar se aprenden dos redes: un *encoder* $q_\phi(z|x)$ y un *decoder* $p_\theta(x|z)$. CompressARC elimina el encoder — el posterior $q(z)$ se aprende **directamente como parámetros `multiposteriors`**, uno por tarea. Esto es posible porque no hay generalización cross-tarea: cada puzzle tiene sus propios μ y σ . El **canal AWGN** que modela cada tensor:

$$z = \sigma_{signal} \cdot \hat{\mu} + \sigma_{noise} \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$
$$KL_i = \frac{1}{2}(\sigma_{noise}^2 + \sigma_{signal}^2 \cdot \hat{\mu}^2 - 1) + \frac{C_{local}}{D}$$

Cada tensor del multitensor decide cuántos bits transmitir. Los `target_capacities` son los parámetros que controlan esa capacidad de canal.

Equivarianzas hardcoded: en la inicialización se aplica `symmetrize_xy` (igual pesos para dims `[...,1,0]` y `[...,0,1]`) y `symmetrize_direction_sharing` (invarianza al grupo diédrico D_4). Esto reduce la longitud de descripción del modelo.

Paso 3 — Optimizador (`analyze_example.py`)

```
optimizer = torch.optim.Adam(model.weights_list, lr=0.01, betas=(0.5, 0.9))
```

Solo se optimizan los **pesos compartidos del modelo** (`weights_list`), no los `multiposteriors` — éstos son parámetros de la tarea específica y se actualizan por separado vía gradient descent sobre μ .

Las elecciones son deliberadas (ver `train.py`):

- **`lr=0.01`** — alta porque el entrenamiento es corto (1500–2000 iter) y parte de Xavier random
- **`β1=0.5`** en vez del clásico 0.9 — reacciona más rápido a cambios de gradiente; crucial en datasets muy pequeños (2–7 ejemplos) donde el gradiente oscila mucho
- **`β2=0.9`** en vez del clásico 0.999 — por la misma razón

Paso 4 — Logger: `solution_selection.Logger` (analyze_example.py)

```
train_history_logger = solution_selection.Logger(task)
```

En `solution_selection.py` el `Logger` inicializa:

```
self.ema_logits      # Media exponencial (decay=0.97) de los logits → candidato EMA
self.current_logits  # Logits del forward pass actual → candidato instantáneo
```

```
self.solution_hashes_count # Dict hash→score acumulado via logaddexp
```

Base teórica (pass@2): la métrica de ARC permite dos intentos. El Logger los materializa así:

- **Candidato 1** (muestra actual): captura la mejor estimación instantánea, ruidosa
- **Candidato 2** (EMA, decay=0.97): suaviza el ruido, converge a la moda del posterior

El score de cada solución candidata en cada paso:

$$\text{score} = -10 \cdot \text{uncertainty} \left[-10 \text{ si } \text{step} < 150 \right] \left[-4 \text{ si EMA} \right]$$

Se acumula con `logaddexp`: una solución que aparece muchas veces con baja incertidumbre acumula score exponencialmente más alto.

Paso 5 — Loop de entrenamiento (analyze_example.py)

```
for train_step in tqdm(range(n_iterations)):
    train.take_step(task, model, optimizer, train_step, train_history_logger)
```

Aquí ocurre **todo el cómputo**. Dentro de `take_step` (train.py):

5a. Forward pass

```
logits, x_mask, y_mask, KL_amounts, KL_names = model.forward()
```

El modelo ejecuta la secuencia por cada uno de los 4 bloques:

```
decode_latents → share_up → softmax → cummax → shift → direction_share → nonlinear → share_down → normalize
```

- **decode_latents**: muestrea $z \sim q_\phi(z)$, calcula KL por tensor, proyecta al residual stream
- **share_up**: propaga info agregada de tensores de bajo rango (sin x/y) hacia tensores más específicos (con x, y)
- **softmax**: selección suave de qué eje atender — genera $2^k - 1$ canales por subconjunto de dims activos
- **cummax**: scan de máximo en 8 direcciones — implementa propagación espacial tipo "extender hasta colisionar"
- **shift**: desplazamiento 1 píxel en cada dirección — comparación entre celdas vecinas
- **direction_share**: 64 matrices lineales con pesos angulares $[1, 0.2, 0.4, 0.2, 1, 0.2, 0.4, 0.2]$ — comunicación entre direcciones respetando D_4
- **nonlinear**: SiLU $x \cdot \sigma(x)$
- **share_down + normalize**: inverso de share_up, estabilización numérica

5b. Pérdida total

La pérdida implementa directamente el **principio MDL** (Liao & Gu 2025, Algoritmo 3):

$$\mathcal{L} = \underbrace{\sum_i \text{KL}(q_\phi(z_i) \parallel \mathcal{N}(0, I))}_{\text{longitud del código latente}} + 10 \cdot \underbrace{H(\text{logits}, \text{pixels})}_{\text{coste de los errores}}$$

```
loss = total_KL + 10 * reconstruction_error
```

El factor 10 es empírico (train.py): sin él el modelo colapsa el posterior ($\mu \rightarrow 0$) y deja de reconstruir.

Para el error de reconstrucción, se usa marginalización sobre offsets de grid posibles mediante `mask_select_logprobs` y `logsumexp` (train.py):

$$\log p(x) = \log \sum_{\text{offset}} p(\text{offset}) \cdot p(x|\text{offset})$$

Esto es especialmente importante cuando el tamaño del output es incierto (`grid_size_uncertain=True`): durante los primeros 100 pasos un curriculum coefficient $0.01^{\max(0, 1-\text{step}/100)}$ suaviza esta penalización para que el modelo primero aprenda colores y luego tamaño.

5c. Backward + step

```
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

Actualiza los 76K parámetros de `weights_list` y los `multiposteriors` de la tarea (están en el grafo computacional, reciben gradientes también).

Paso 6 — Visualización periódica ([analyze_example.py](#))

```
if (train_step+1) % 50 == 0:
    visualization.plot_solution(train_history_logger, fname=..., task_name=task_name)
```

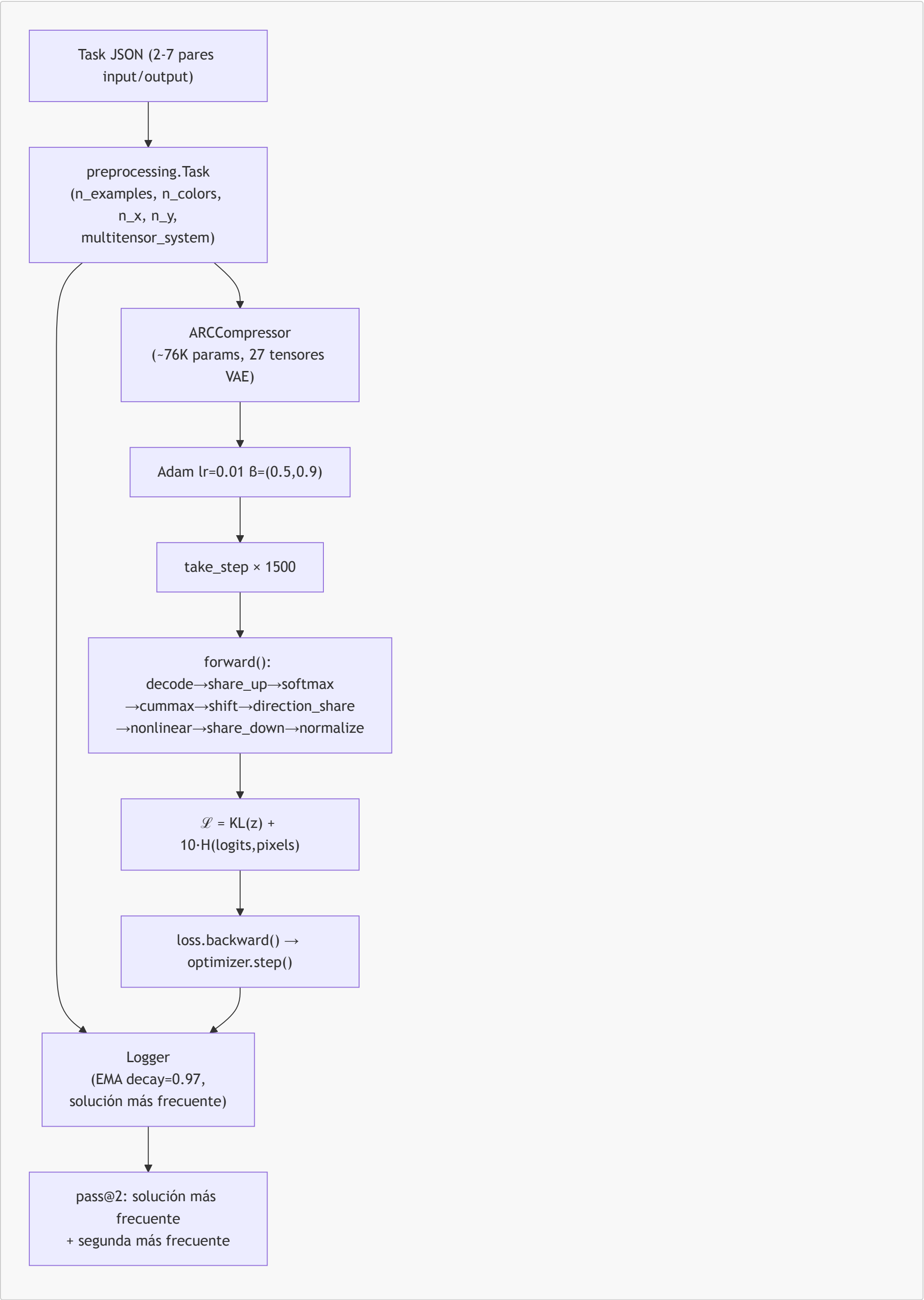
Cada 50 pasos se guarda una imagen del estado actual del candidato más frecuente. Permite ver cómo la red converge (o colapsa) visualmente.

Paso 7 — Guardado ([analyze_example.py](#))

```
np.savez(folder + task_name + '_KL_curves.npz',
         KL_curves=...,
         reconstruction_error_curve=...,
         multiposteriors=model.multiposteriors,
         target_capacities=model.target_capacities,
         decode_weights=model.decode_weights)
```

Se guardan las curvas de KL por tensor (diagnóstico de **posterior collapse** — si casi todos los tensores caen a $KL \approx 0$ antes del paso 100, la run probablemente falla), las curvas de error de reconstrucción, y los parámetros aprendidos de la tarea específica.

Visión de conjunto del flujo



El invariante MDL que conecta todo: minimizar $\mathcal{L}$ es equivalente a buscar la descripción más corta de los pares de entrenamiento. Si esa descripción es genuinamente compacta (KL baja + reconstrucción perfecta), el modelo ha descubierto la **regla latente** de la tarea — y esa misma regla, aplicada a través de los

mismos pesos, predice el output del ejemplo de test.

Plan: Cache Phase 1 VRAM measurements

Two additions to parallel_train.py only.

1 — Add these 3 functions before the run_split function (line ~233)

```
# — Phase 1 memory-measurement cache —————

def _cache_path(split):
    return f'memory_cache_{split}.json'

def _gpu_fingerprint(n_gpus):
    return {
        'n_gpus': n_gpus,
        'gpu_names': [torch.cuda.get_device_name(i) for i in range(n_gpus)],
        'gpu_vram_total_gb': [
            round(torch.cuda.mem_get_info(i)[1] / 1024**3, 2)
            for i in range(n_gpus)
        ],
        'torch_version': torch.__version__,
    }

def load_memory_cache(split, n_gpus, required_task_names):
    path = _cache_path(split)
    if not os.path.exists(path):
        return None
    try:
        with open(path, 'r') as f:
            cache = json.load(f)
        if cache.get('fingerprint') != _gpu_fingerprint(n_gpus):
            return None
        measurements = cache.get('measurements', {})
        if not all(name in measurements for name in required_task_names):
            return None
        return {name: int(measurements[name]) for name in required_task_names}
    except Exception:
        return None

def save_memory_cache(split, n_gpus, memory_dict):
    path = _cache_path(split)
    cache = {
        'fingerprint': _gpu_fingerprint(n_gpus),
        'created_at': time.strftime('%Y-%m-%d %H:%M:%S'),
        'measurements': {k: int(v) for k, v in memory_dict.items()},
    }
    with open(path, 'w') as f:
        json.dump(cache, f, indent=2)
```

2 — Replace the Phase 1 block inside run_split() (~line 270)

Replace this:

```
# — Phase 1: measure VRAM footprint (2 iterations per task) —————
arc_logger.log_phase(...)
gpu_memory_quotas = [...]
gpu_task_quotas = [...]
memory_dict, _, _, t_p1 = parallelize_runs(...)
arc_logger.info(f'Phase 1 complete in {t_p1:.1f}s')
```

With this:

```
# — Phase 1: measure VRAM footprint (or load from cache) —————
gpu_memory_quotas = [torch.cuda.mem_get_info(i)[0] for i in range(n_gpus)]

cached = load_memory_cache(split, n_gpus, original_task_names)
if cached is not None:
```

```

arc_logger.log_phase(
    f'Phase 1 - SKIPPED - loaded from {_cache_path(split)}'
)
arc_logger.info(
    f'Cache valid for {len(cached)} tasks. '
    f'Delete {_cache_path(split)} to force re-measurement.'
)
memory_dict = cached
else:
    arc_logger.log_phase(
        f'Phase 1 - Memory measurement (2 iterations × {n_tasks} tasks)'
    )
    gpu_task_quotas = [int(q // (4 * 1024**3)) for q in gpu_memory_quotas]
    memory_dict, _, _, t_p1 = parallelize_runs(
        gpu_task_quotas, [1] * n_tasks, 2,
        original_task_names, split, n_tasks, n_gpus, n_cpus,
        arc_logger=arc_logger, n_original_tasks=n_tasks,
        quiet=True, verbose=True,
    )
    save_memory_cache(split, n_gpus, memory_dict)
    arc_logger.info(
        f'Phase 1 complete in {t_p1:.1f}s - saved to {_cache_path(split)}'
    )

```

How it works:

- First run: Phase 1 runs normally, saves `memory_cache_training.json` with GPU fingerprint (name + VRAM size + PyTorch version)
- Every subsequent run: loads the file, compares fingerprint → if it matches, skips Phase 1 entirely (saves 5–15 min)
- Cache is automatically invalidated if you change GPU or PyTorch version
- To force re-measurement: `rm memory_cache_training.json`

Completed: *Implement caching for Phase 1 VRAM measurements* (1/3)